

Claude Code / claude.ai / チャット対応

Jen v3 使い方ガイド

Fable Edition — 指揮はFable、実行は安く

ゴールを渡すだけで、分解・委譲・検証・差し戻し・リリース準備まで
PMO型オーケストレーター「Jen」が管理する。

v3 では指揮系を Claude Fable 5 に載せ替え、18体のエージェントで運用する。

fable

jen-pmo / jen-deep-solver

opus

architect / debugger / strict-verifier

sonnet

builder / frontend / test / research / reviewers

haiku

scout

1. Jen v3 とは

Claude Code / claude.ai / チャット対応

Jen v3 使い方ガイド

Fable Edition 指標はFable 実行はAI

項目	v2	v3
jen-pmo のモデル	sonnet	fable (effort: max)
昇格ラダー	haiku → sonnet → opus	haiku → sonnet → opus → fable
最終昇格先	(人間へ)	jen-deep-solver (fable) を新設 → 人間へ
longrun	3ループ修復	多日自走前提+フォールバック復帰+コスト自己監査
nested subagents	なし	実装系が read-only scout (haiku) を子に持てる
エージェント数	17体	18体

2. 3つの使いかた (どれか選ぶ?)

fable jen-pmo / jen-deep-solver		
形態	向いている人	できること
A. Claude Code パック (推奨)	リポジトリで本格運用したい	18体のsubagent並列、hooks、worktree、品質ゲート、longrun 全機能
B. claude.ai スキル (ポータブル)	ブラウザ/アプリで使いたい	単一コンテキストでの PMO 運用。skills-portable/ の .skill をアップロード
C. チャット貼り付け (最速)	今すぐ試したい	jen-activate-prompt.txt を新規チャットに貼るだけの簡易版

3. セットアップ (A: Claude Code)

```
claude update # 1) Claude Code ■■■
cd <your-repo> # 2) ■■■■■■■■
unzip claude-code-pack/jen-v3-claude-code-pack-ready.zip
./install.sh # 3) ■■■dir■■■■■■.gitignore ■■■
unset CLAUDE_CODE_SUBAGENT_MODEL # 4) ■■■: ■■■■■■■■
# ■■■: hooks ■■■■■■■■
cp .claude/settings.jen.example.json .claude/settings.json
```

Claude Code を開く
/model fable → メインセッションを Fable 5 に
/agents → jen-* (jen-deep-solver 含む18体) が見えるか確認
/jen conduct → 最初のゴールを渡す

Claude Code / claude.ai / チャット対応

Jen v3 使い方ガイド

Fable Edition — 指揮はFable、実行は安く

ゴールを渡すだけで、分解・委譲・検証・差し戻し・リリース準備まで
PMO型オーケストレーター「Jen」が管理する。
v3 では指揮系を Claude Fable 5 に載せ替え、18体のエージェントで運用する。

fable	jen-pmo / jen-deep-solver
opus	architect / debugger / strict-verifier
sonnet	builder / frontend / test / research / reviewers
haiku	scout

4. 6つのモード

モード	使いどころ	動き
route	小さなタスク（1〜5工程）	1工程ずつ担当を選び、verifier 合格まで回す
conduct	中〜大規模タスク	ゴールを依存グラフ（DAG）へ分解。独立工程は並列、同一ファイルは直列
repair	失敗・バグから開始	再現固定 → 分類 → debugger/test へ委譲 → 最小修正 → 回帰 → 検収
review	検収・リスク洗い出し	UX / セキュリティ / 反対意見 / 収益化の各レビュアーを招集
release	PR・リリース準備	PR要約・検証パケット・ロールバック手順。deploy は人間承認で停止
longrun	長時間自走	サイクル毎に Mission→Task→実装→検証→Checkpoint→Handoff を反復

Fable Edition — 指揮はFable、実行は女く

/jen conduct
Goal: サインアップ後のオンボーディング画面を改善し、初回ユーザーが3分以内に主要機能へ到達できるようにする。
Must Have:
- 既存デザイントークンに合わせる / モバイル対応 / 既存テストを壊さない
Out of Scope:
- 課金導線の変更 / DB schema 変更
Acceptance Criteria:
- 初回画面に次アクションが表示される
- 空状態とエラー状態がある
- lint / typecheck / test が通る

fable
Have → Out of Scope・Acceptance Criteria の4点を渡すと精度が大きくなります。曖昧なまま渡した場合、品質向上とつながり mission brief を作ります。

opus architect / debugger / strict-verifier

5. エージェント一覧とモデル階層

sonnet builder / frontend / test / research / reviewers

エージェント	model	役割
jen-pmo	fable	指揮。Mission Brief / 台帳 / 委譲 / 品質ゲート管理。自分では実装しない
jen-scout	haiku	リポジトリ内の下調べ・要約（read-only）
jen-research	sonnet	外部・公式ドキュメント調査
jen-builder / jen-frontend	sonnet	通常実装 / UI実装
jen-test	sonnet	テスト・QA観点
jen-product-strategist / jen-ideation	sonnet	仕様改善・優先度 / 発想出し
jen-ux-critic / jen-contrarian-reviewer	sonnet	UI/UX検収 / 反対意見・破綻予測
jen-security-reviewer / jen-monetization-reviewer	sonnet	security・auth・payment / 収益化導線
jen-architect	opus	難設計・境界・性能
jen-debugger	opus	原因不明バグ
jen-verifier / jen-strict-verifier	sonnet / opus	通常検収 / 高リスク検収（ACCEPT/REJECTのみ返す）
jen-release-manager	sonnet	PR・リリース準備
jen-deep-solver	fable	最終昇格先。opus層が失敗した問題のみ（v3新設）

6. 昇格ラダーと Human Gate

haiku → sonnet → opus → fable(jen-deep-solver) → 人間へ返す

verifier REJECT 1回 : 同じ担当へ差し戻し

verifier REJECT 2回 : opus 層 (architect / debugger) へ昇格

REJECT 3回 or opus層失敗 : jen-deep-solver (fable) へ昇格

deep-solver 失敗 / strict-verifier REJECT : Human Gate

Claude Code / claude.ai / チャット対応

Jen v3 使い方ガイド

Fable Edition — 指揮はFable、実行は安く

ゴールを渡すだけで、分解・委譲・検証・差し戻し・リリース準備まで
PMO型オーケストレーター「Jen」が管理する。

v3 では指揮系を Claude Fable 5 に載せ替え、18体のエージェントで運用する。

fable	jen-pmo / jen-deep-solver
opus	architect / debugger / strict-verifier
sonnet	builder / frontend / test / research / reviewers
haiku	scout

7. longrun 運用 (Fable 多日自走)

ファイル	内容
mission.md	Goal / Non-goals / 制約 / AC / Human Gates
tasks.json	タスク台帳（ID・状態・担当・依存・証跡）
assumptions.md / decisions.md	仮定の記録 / 設計判断の理由
verification.md	検証コマンドと結果の証跡
ideas.md	スコープ外の提案置き場（勝手に実装しない）
handoff.md	中断・再開用の要約。毎サイクル更新

Fable Edition — 指揮はFable、実行は安く

ゴールを渡すだけで、分解・委譲・検証・差し戻し・リリース準備まで

PMO型オーケストレーター「Jen」が管理する

v3では指揮系を Claude Fable 5 に載せ替え、18体のエージェントで運用する。

TableのAPI単価はOpus 4.8の約2倍のため、この規律が長時間運用のコストを決めます。

8. claude.ai / チャットでの使いた (R・C)

opus / architect / debugger / strict-verifier

builder / frontend / test / research / reviewers

(スキル) から1つずつアップロードします。会話で「ジェン、～して」と呼ぶか、モード名 (conduct
haiku scout
ールを渡すと起動します。モデルは Fable 5 を推奨。

9. トラブルシューティング / FAQ

症状 / 質問	対処
/model fable が効かない、fable が選べない	claude update で v2.1.170+ へ更新。プラン/クレジットで Fable 5 が有効か確認
fable 指定の agent が別モデルで動いている	CLAUDE_CODE_SUBAGENT_MODEL を unset (frontmatter を上書きするため)
/agents に jen-* が出ない	リポジトリ直下で展開したか確認。 .claude/agents/ の存在を確認して Claude Code を再起動
セッションが Opus 4.8 に切り替わった	\$ 7 の復帰手順 (handoff → 新セッション → /model fable → 再開)

症状 / 質問	対処
コストが想定より速く減る	PMO が自分で実装していないか確認（drift）。deep-solver 直行が起きていないか台帳を確認
hooks が動かない	settings.jen.example.json を settings.json にコピーしたか確認（デフォルトOFF）
Jen が勝手に機能を追加する	仕様外の提案は .jen/ideas.md 行きがルール。混入したら review モードで指摘し差し戻す
v2 から移行できる？	可能。.jen/ 状態ファイルはそのまま。パックを上書き展開し install.sh を再実行

Claude Code / claude.ai / チャット対応

約束ごと（全モード共通）：

ログ・レポートは日本語／コードと識別子は英語。過剰主張しない（未完を「完成」と言わない）。検証は省略しない。出力は簡潔。

Jen v3 使い方ガイド

Fable Edition — 指揮はFable、実行は安く

ゴールを渡すだけで、分解・委譲・検証・差し戻し・リリース準備まで PMO型オーケストレーター「Jen」が管理する。
v3 では指揮系を Claude Fable 5 に載せ替え、18体のエージェントで運用する。

fable	jen-pmo / jen-deep-solver
opus	architect / debugger / strict-verifier
sonnet	builder / frontend / test / research / reviewers
haiku	scout